Version: 1.0



Selection

C-Listes-chaînées

Résumé

Implémente des fonctions permettant de créer, modifier et manipuler des listes simplement chaînées en C, en gérant les opérations d'insertion, de suppression et de tri.

#C

#Structures-de-données

#Listes-chaînées



Avertissement relatif à la propriété intellectuelle

L'ensemble du contenu présenté dans ce module de formation — y compris, sans s'y limiter, les textes, images, graphiques et autres éléments — est protégé par les droits de propriété intellectuelle détenus par l'Association 42.

Conditions d'utilisation

- **Usage personnel :** L'utilisation du contenu de ce module est strictement réservée à un usage personnel. Toute reproduction, diffusion, modification, utilisation publique ou commerciale est interdite sans l'autorisation écrite préalable de l'Association 42.
- **Respect de l'intégrité :** Il est interdit de modifier, altérer ou adapter le contenu de manière à en dénaturer le sens ou à porter atteinte à son intégrité.

Protection des droits

Toute violation des présentes conditions constitue une atteinte aux droits de propriété intellectuelle et peut faire l'objet de poursuites. L'Association 42 se réserve le droit d'engager toute action nécessaire à la protection de ses droits, notamment par voie judiciaire et en réclamant des dommages et intérêts le cas échéant.

Pour toute question relative à l'utilisation du contenu ou pour obtenir une autorisation, veuillez contacter : `legal@42.fr`

Table des matières

1 Instructions 1

2 Préambule 5

3 Informations pour ce projet 7

4 Exercice 0 : ft_create_elem 8

5 Exercice 1 : ft_list_push_front 9

6 Exercice 2 : ft_list_size 10

7 Exercice 3 : ft_list_last 11

8 Exercice 4 : ft_list_push_back 12

9 Exercice 5 : ft_list_push_strs 13

10 Exercice 6 : ft_list_clear 14

11 Exercice 7 : ft_list_at 15

12 Exercice 8 : ft_list_reverse 16

13 Exercice 9 : ft_list_foreach 17

14 Exercice 10 : ft_list_foreach_if 18

15 Exercice 11 : ft_list_find 19

16 Exercice 12 : ft_list_remove_if 20

17 Exercice 13 : ft_list_merge 21

18 Exercice 14 : ft_list_sort 22

19 Exercice 15 : ft_list_reverse_fun 23

20 Exercice 16 : ft_sorted_list_insert 24

21 Exercice 17 : ft_sorted_list_merge 25

22 Soumission et évaluation par les pairs 26

Chapitre 1

Instructions

- Seule cette page servira de référence : ne faites pas confiance aux rumeurs.
- Attention ! Ce document pourrait potentiellement changer jusqu'à la soumission.
- Assurez-vous d'avoir les permissions appropriées sur vos fichiers et répertoires.
- Vous devez suivre les procédures de soumission pour tous vos exercices.
- Vos exercices seront vérifiés et notés par vos camarades de classe.
- De plus, vos exercices seront vérifiés et notés par un programme appelé Moulinette.
- Moulinette est très méticuleuse et stricte dans son évaluation de votre travail. Elle est entièrement automatisée, et il n'y a aucun moyen de négocier avec elle. Donc, pour éviter les mauvaises surprises, soyez aussi minutieux que possible.
- Moulinette n'est pas très ouverte d'esprit. Elle n'essayera pas de comprendre votre code s'il ne respecte pas la Norme. Moulinette s'appuie sur un programme appelé `norminette` pour vérifier si vos fichiers respectent la norme. En bref : il serait stupide de soumettre un travail qui ne passe pas la vérification de `norminette`.
- Ces exercices sont soigneusement disposés par ordre de difficulté - du plus facile au plus difficile. Nous `ne considérerons pas` un exercice plus difficile réussi si un exercice plus facile n'est pas parfaitement fonctionnel.
- Utiliser une fonction interdite est considéré comme de la tricherie. Les tricheurs obtiennent -42, et cette note est non négociable.
- Vous n'aurez à soumettre une fonction main() que si nous demandons un programme.
- Moulinette compile avec ces flags : -Wall -Wextra -Werror, et utilise `cc`.
- Si votre programme ne compile pas, vous obtiendrez `0`.
- Vous ne pouvez pas laisser aucun fichier supplémentaire dans votre répertoire autre que ceux spécifiés dans le sujet.
- Vous avez une question ? Demandez à votre pair à droite. Sinon, essayez votre pair à gauche.
- Votre guide de référence s'appelle `Google / man / Internet / ...`.
- Consultez le Slack Piscine.
- Examinez attentivement les exemples. Ils pourraient très bien appeler à des détails qui ne sont pas explicitement mentionnés dans le sujet...

© 2026 Association 42. - Tous droits réservés

- Par Odin, par Thor ! Utilisez votre cerveau ! ! !



N'oubliez pas d'ajouter l'*en-tête standard 42* dans chacun de vos fichiers .c/.h. Norminette vérifie son existence de toute façon !



Norminette doit être lancée avec le flag *-R CheckForbiddenSourceHeader*. Moulinette l'utilisera aussi.

© 2026 Association 42. - Tous droits réservés

Contexte

La Piscine C est intense. C'est votre premier grand défi à 42 — une plongée profonde dans la résolution de problèmes, l'autonomie et la communauté.

Durant cette phase, votre objectif principal est de construire vos fondations — à travers la lutte, la répétition, et surtout l'échange d'**apprentissage par les pairs**.

À l'ère de l'IA, les raccourcis sont faciles à trouver. Cependant, il est important de considérer si votre utilisation de l'IA vous aide vraiment à grandir — ou si elle se contente de vous empêcher de développer de vraies compétences.

La Piscine est aussi une expérience humaine — et pour l'instant, rien ne peut remplacer cela. Pas même l'IA.

Pour un aperçu plus complet de notre position sur l'IA — en tant qu'outil d'apprentissage, dans le cadre du programme ICT, et en tant qu'attente croissante sur le marché du travail — veuillez vous référer à la FAQ dédiée disponible sur l'intranet.

Message principal

- Construire des fondations solides sans raccourcis.
- Développer réellement des compétences techniques et personnelles.
- Expérimenter un véritable apprentissage par les pairs, commencer à apprendre à apprendre et à résoudre de nouveaux problèmes.
- Le parcours d'apprentissage est plus important que le résultat.
- Apprendre les risques associés à l'IA, et développer des pratiques de contrôle efficaces et des contre-mesures pour éviter les pièges courants.

Règles pour les apprenants :

- Vous devez appliquer le raisonnement à vos tâches assignées, surtout avant de vous tourner vers l'IA.
- Vous ne devez pas demander de réponses directes à l'IA.
- Vous devez apprendre l'approche globale de 42 sur l'IA.

© 2026 Association 42. - Tous droits réservés

Résultats de la phase :

Dans cette phase de fondation, vous obtiendrez les résultats suivants :

- Acquérir des bases techniques et de codage appropriées.
- Savoir pourquoi et comment l'IA peut être dangereuse pendant cette phase.

Commentaires et exemple :

- Oui, nous savons que l'IA existe — et oui, elle peut résoudre vos activités. Mais vous êtes ici pour apprendre, pas pour prouver que l'IA a appris. Ne gaspillez pas votre temps (ou le nôtre) juste pour démontrer que l'IA peut résoudre le problème donné.
- Apprendre à 42 ne consiste pas à connaître la réponse — il s'agit de développer la capacité à en trouver une. L'IA vous donne la réponse directement, mais cela vous empêche de construire votre propre raisonnement. Et le raisonnement prend du temps, des efforts, et implique des échecs. Le chemin vers le succès n'est pas censé être facile.
- Gardez à l'esprit que lors des examens, l'IA n'est pas disponible — pas d'internet, pas de smartphones, etc. Vous réaliserez rapidement si vous avez trop compté sur l'IA dans votre processus d'apprentissage.
- L'apprentissage par les pairs vous expose à différentes idées et approches, améliorant vos compétences interpersonnelles et votre capacité à penser de manière divergente. C'est bien plus précieux que de simplement discuter avec un bot. Alors ne soyez pas timide — parlez, posez des questions, et apprenez ensemble !
- Oui, l'IA fera partie du programme — à la fois comme outil d'apprentissage et comme sujet en soi. Vous aurez même l'occasion de construire votre propre logiciel d'IA. Afin d'en apprendre davantage sur notre approche crescendo, vous passerez par la documentation disponible sur l'intranet.

✓ Bonne pratique :

> Je suis bloqué sur un nouveau concept. Je demande à quelqu'un à proximité comment il l'a abordé. Nous parlons pendant 10 minutes — et soudain, ça fait clic. Je comprends.

✗ Mauvaise pratique :

> J'utilise secrètement l'IA, je copie du code qui semble correct. Pendant l'évaluation par les pairs, je ne peux rien expliquer. J'échoue. Pendant l'examen — sans IA — je suis à nouveau bloqué. J'échoue.

© 2026 Association 42. - Tous droits réservés

Chapitre 2

Préambule

SPOILER ALERT
NE LIS PAS LA PAGE SUIVANTE

© 2026 Association 42. - Tous droits réservés

Tu as été prévenu.

- Dans `Star Wars`, Dark Vador est le père de Luke.
- Dans `Usual Suspects`, Verbal est Keyser Söze.
- Dans `Fight Club`, Tyler Durden et le narrateur sont la même personne.
- Dans `Sixième Sens`, Bruce Willis est mort depuis le début.
- Dans `Les Autres`, les habitants de la maison sont des fantômes, et inversement.
- Dans `Bambi`, la mère de Bambi meurt.
- Dans `The Village`, les monstres sont les villageois et le film se passe à notre époque.
- Dans `Harry Potter`, Dumbledore meurt.
- Dans `La Planète des Singes`, le film se déroule sur Terre.
- Dans `Game of Thrones`, Robb Stark et Joffrey Baratheon meurent le jour de leur mariage.
- Dans `Twilight`, les vampires brillent au soleil.
- Dans `Stargate SG-1, Saison 1, Épisode 18`, O'Neill et Carter sont en Antarctique.
- Dans `The Dark Knight Rises`, Miranda Tate est Talia Al'Ghul.
- Dans `Super Mario Bros`, la princesse est dans un autre château.

© 2026 Association 42. - Tous droits réservés

Chapitre 3

Informations pour ce projet

Pour les exercices suivants, vous devez utiliser la structure suivante :

ft_list.h

```
typedef struct          s_list
{
        struct s_list   *next;
        void            *data;
}                       t_list;
```

Vous devrez inclure cette structure dans un fichier `ft_list.h` et le rendre pour chaque exercice.

À partir de l'exercice 01, nous utiliserons notre propre ft_create_elem, donc prévoyez-le (il pourrait être utile d'avoir son prototype dans ft_list.h...).

© 2026 Association 42. - Tous droits réservés

Chapitre 4

Exercice 0 : ft_create_elem

	Exercice: 0	
ft_create_elem		
Répertoire: *ex0/*		
Fichiers à Rendre: `ft_create_elem.c, ft_list.h`		
Autorisé: `malloc`		

- Créez la fonction `ft_create_elem` qui crée un nouvel élément de type `t_list`.
- Elle doit affecter `data` à l'argument donné et `next` à NULL.

Prototype :

```
t_list *ft_create_elem(void *data);
```

© 2026 Association 42. - Tous droits réservés

Chapitre 5

Exercice 1 : ft_list_push_front

	Exercice: 1	
ft_list_push_front		
Répertoire: *ex1/*		
Fichiers à Rendre: `ft_list_push_front.c, ft_list.h`		
Autorisé: `ft_create_elem`		

- Créez la fonction `ft_list_push_front` qui ajoute un nouvel élément de type `t_list` au début de la liste.
- Elle doit affecter `data` à l'argument donné.
- Si nécessaire, elle mettra à jour le pointeur de début de liste.

Prototype :

```
void ft_list_push_front(t_list **begin_list, void *data);
```

© 2026 Association 42. - Tous droits réservés

Chapitre 6

Exercice 2 : ft_list_size

	Exercice: 2	
ft_list_size		
Répertoire: *ex2/*		
Fichiers à Rendre: `ft_list_size.c, ft_list.h`		
Autorisé: `None`		

- Créez la fonction `ft_list_size` qui retourne le nombre d'éléments dans la liste.

Prototype :

```
int ft_list_size(t_list *begin_list);
```

© 2026 Association 42. - Tous droits réservés

Chapitre 7

Exercice 3 : ft_list_last

	Exercice: 3	
ft_list_last		
Répertoire: *ex3/*		
Fichiers à Rendre: `ft_list_last.c, ft_list.h`		
Autorisé: `None`		

- Créez la fonction `ft_list_last` qui retourne le dernier élément de la liste.

Prototype :

```
t_list *ft_list_last(t_list *begin_list);
```

© 2026 Association 42. - Tous droits réservés

Chapitre 8

Exercice 4 : ft_list_push_back

	Exercice: 4	
ft_list_push_back		
Répertoire: *ex4/*		
Fichiers à Rendre: `ft_list_push_back.c, ft_list.h`		
Autorisé: `ft_create_elem`		

- Créez la fonction `ft_list_push_back` qui ajoute un nouvel élément de type `t_list` à la fin de la liste.
- Elle doit attribuer à `data` l'argument fourni.
- Si nécessaire, elle doit mettre à jour le pointeur en début de liste.

Prototype :

```
void ft_list_push_back(t_list **begin_list, void *data);
```

© 2026 Association 42. - Tous droits réservés

Chapitre 9

Exercice 5 : ft_list_push_strs

	Exercice: 5	
	ft_list_push_strs	
	Répertoire: *ex*5/	
	Fichiers à Rendre: `ft_list_push_strs.c, ft_list.h`	
	Autorisé: `ft_create_elem`	

- Créez la fonction `ft_list_push_strs` qui crée une nouvelle liste contenant toutes les chaînes pointées par les éléments de `strs`.
- `size` est la taille de `strs`.
- Le premier élément doit se trouver à la fin de la liste.
- L'adresse du premier maillon de la liste est retournée.

Prototype :

```
t_list *ft_list_push_strs(int size, char **strs);
```

© 2026 Association 42. - Tous droits réservés

Chapitre 10

Exercice 6 : ft_list_clear

	Exercice: 6	
	ft_list_clear	
	Répertoire: *ex6/*	
	Fichiers à Rendre: `ft_list_clear.c, ft_list.h`	
	Autorisé: `free`	

- Créez la fonction `ft_list_clear` qui supprime et libère tous les maillons de la liste.
- `free_fct` est utilisée pour libérer chaque `data`.

Prototype :

```
void ft_list_clear(t_list *begin_list, void (*free_fct)(void
*));
```

© 2026 Association 42. - Tous droits réservés

Chapitre 11

Exercice 7 : ft_list_at

	Exercice: 7	
ft_list_at		
Répertoire: *ex7/*		
Fichiers à Rendre: `ft_list_at.c, ft_list.h`		
Autorisé: `None`		

- Créez la fonction `ft_list_at` qui retourne le Nième élément de la liste, en considérant que le premier élément correspond à `nbr` égal à 0.
- En cas d'erreur, elle doit retourner un pointeur nul.

Prototype :

```
t_list *ft_list_at(t_list *begin_list, unsigned int nbr);
```

© 2026 Association 42. - Tous droits réservés

Chapitre 12

Exercice 8 : ft_list_reverse

	Exercice: 8	
ft_list_reverse		
Répertoire: *ex8/*		
Fichiers à Rendre: `ft_list_reverse.c`		
Autorisé: `None`		

- Créez la fonction `ft_list_reverse` qui inverse l'ordre des éléments d'une liste. La valeur de chaque élément doit rester la même.
- Attention, dans cette fonction, nous utiliserons notre propre fichier `ft_list.h`.

Prototype :

```
void ft_list_reverse(t_list **begin_list);
```

© 2026 Association 42. - Tous droits réservés

Chapitre 13

Exercice 9 : ft_list_foreach

	Exercice: 9	
ft_list_foreach		
Répertoire: *ex9/*		
Fichiers à Rendre: `ft_list_foreach.c, ft_list.h`		
Autorisé: `None`		

- Créez la fonction `ft_list_foreach` qui applique la fonction donnée en argument à chacun des éléments de la liste.
- `f` doit être appliquée dans le même ordre que la liste.

Prototype :

```
void ft_list_foreach(t_list *begin_list, void (*f)(void *));
```

- La fonction pointée par `f` sera utilisée comme suit :

Utilisation :

```
(*f)(list_ptr->data);
```

© 2026 Association 42. - Tous droits réservés

Chapitre 14

Exercice 10 : ft_list_foreach_if

	Exercice: 10	
ft_list_foreach_if		
Répertoire: *ex*10/		
Fichiers à Rendre: `ft_list_foreach_if.c, ft_list.h`		
Autorisé: `None`		

- Créez la fonction `ft_list_foreach_if` qui applique la fonction donnée en argument à certains éléments de la liste.
- Appliquez la fonction uniquement aux éléments pour lesquels `cmp` avec `data_ref` retourne 0.
- `f` doit être appliquée dans le même ordre que la liste.

Prototype :

```
void ft_list_foreach_if(t_list *begin_list, void (*f)(void *),
void *data_ref, int (*cmp)(void *, void *));
```

- Les fonctions pointées par `f` et `cmp` seront utilisées comme suit :

Utilisation :

```
 (*f)(list_ptr->data);
(*cmp)(list_ptr->data, data_ref);
```



Par exemple, la fonction `cmp` pourrait être `ft_strcmp`...

© 2026 Association 42. - Tous droits réservés

Chapitre 15

Exercice 11 : ft_list_find

	Exercice: 11	
	ft_list_find	
	Répertoire: *ex*11/	
	Fichiers à Rendre: `ft_list_find.c, ft_list.h`	
	Autorisé: `None`	

- Créez la fonction `ft_list_find` qui retourne l'adresse du premier élément dont les données, comparées à `data_ref` avec `cmp`, font que `cmp` retourne 0.

Prototype :

```
t_list *ft_list_find(t_list *begin_list, void *data_ref, int
(*cmp)(void *, void *));
```

- La fonction pointée par `cmp` sera utilisée comme suit :

Utilisation :

```
(*cmp)(list_ptr->data, data_ref);
```

© 2026 Association 42. - Tous droits réservés

Chapitre 16

Exercice 12 : ft_list_remove_if

	Exercice: 12	
ft_list_remove_if		
Répertoire: *ex*12/		
Fichiers à Rendre: `ft_list_remove_if.c, ft_list.h`		
Autorisé: `free`		

- Créez la fonction `ft_list_remove_if`, qui supprime de la liste tous les éléments dont les données, lorsqu'elles sont comparées à `data_ref` avec `cmp`, font que `cmp` retourne 0.
- Les données des éléments à supprimer doivent être libérées à l'aide de `free_fct`.

Prototype :

```
void ft_list_remove_if(t_list **begin_list, void *data_ref,
int (*cmp)(void *, void *), void (*free_fct)(void *));
```

- Les fonctions pointées par `cmp` et `free_fct` seront utilisées comme suit :

Utilisation :

```
(*cmp)(list_ptr->data, data_ref);
(*free_fct)(list_ptr->data);
```

© 2026 Association 42. - Tous droits réservés

Chapitre 17

Exercice 13 : ft_list_merge

	Exercice: 13	
ft_list_merge		
Répertoire: *ex*13/		
Fichiers à Rendre: `ft_list_merge.c, ft_list.h`		
Autorisé: `None`		

- Créez la fonction `ft_list_merge`, qui ajoute tous les éléments de la liste `begin_list2` à la fin de la liste `begin_list1`.
- La création de nouveaux éléments n'est pas autorisée.

Prototype :

```
void ft_list_merge(t_list **begin_list1, t_list *begin_list2);
```

© 2026 Association 42. - Tous droits réservés

Chapitre 18

Exercice 14 : ft_list_sort

	Exercice: 14	
ft_list_sort		
Répertoire: *ex*14/		
Fichiers à Rendre: `ft_list_sort.c, ft_list.h`		
Autorisé: `None`		

- Créez la fonction `ft_list_sort`, qui trie les éléments de la liste par ordre croissant en comparant leurs données à l'aide d'une fonction.

Prototype :

```
void ft_list_sort(t_list **begin_list, int (*cmp)(void *, void *));
```

- La fonction pointée par `cmp` sera utilisée comme suit :

Utilisation :

```
(*cmp)(list_ptr->data, list_other_ptr->data);
```



`cmp` pourrait être, par exemple, `ft_strcmp`.

© 2026 Association 42. - Tous droits réservés

Chapitre 19

Exercice 15 : ft_list_reverse_fun

	Exercice: 15	
ft_list_reverse_fun		
Répertoire: *ex*15/		
Fichiers à Rendre: `ft_list_reverse_fun.c, ft_list.h`		
Autorisé: `None`		

- Créez la fonction `ft_list_reverse_fun`, qui inverse l'ordre des éléments de la liste.

Prototype :

```
void ft_list_reverse_fun(t_list *begin_list);
```

© 2026 Association 42. - Tous droits réservés

Chapitre 20

Exercice 16 : ft_sorted_list_insert

	Exercice: 16	
ft_sorted_list_insert		
Répertoire: *ex*16/		
Fichiers à Rendre: `ft_sorted_list_insert.c, ft_list.h`		
Autorisé: `ft_create_elem`		

- Créez la fonction `ft_sorted_list_insert`, qui crée un nouvel élément et l'insère dans une liste tout en garantissant que la liste reste triée dans l'ordre croissant.

Prototype :

```
void ft_sorted_list_insert(t_list **begin_list, void *data, int
(*cmp)(void *, void *));
```

- La fonction pointée par `cmp` sera utilisée comme suit :

Utilisation :

```
(*cmp)(list_ptr->data, list_other_ptr->data);
```

© 2026 Association 42. - Tous droits réservés

Chapitre 21

Exercice 17 : ft_sorted_list_merge

	Exercice: 17	
ft_sorted_list_merge		
Répertoire: *ex*17/		
Fichiers à Rendre: `ft_sorted_list_merge.c, ft_list.h`		
Autorisé: `None`		

- Créez la fonction `ft_sorted_list_merge`, qui intègre les éléments d'une liste triée `begin_list2` dans une autre liste triée `begin_list1`, en garantissant que `begin_list1` reste triée dans l'ordre croissant.

Prototype :

```
void ft_sorted_list_merge(t_list **begin_list1, t_list *begin_list2,
int (*cmp)(void *, void *));
```

- La fonction pointée par `cmp` sera utilisée comme suit :

Utilisation :

```
(*cmp)(list_ptr->data, list_other_ptr->data);
```

© 2026 Association 42. - Tous droits réservés

Chapitre 22

Soumission et évaluation par les pairs

Rendez votre devoir dans votre dépôt `Git` comme d'habitude. Seul le contenu présent dans votre dépôt sera évalué lors de la soutenance. N'hésitez pas à vérifier deux fois les noms de vos fichiers pour vous assurer qu'ils sont corrects.



Vous devez rendre uniquement les fichiers demandés par l'énoncé de ce projet.

© 2026 Association 42. - Tous droits réservés